Stock Market Anomaly Detection

GME Analysis Using Unsupervised Machine Learning

Project Report

Academic Year 2025−**2**6

Project Team

Member 1: VEDANT MISHRA

Member 2: HARSH GHARAT

Member 3: MOHIT MADIWALA

Primary Asset:	GameStop Corporation (GME)
Tech Stack:	Python, scikit-learn, TensorFlow/Keras, Streamlit, Plotly
Approach:	Unsupervised Anomaly Detection (5 methods)
Deployment:	Interactive Streamlit Web Application

Abstract

This project implements an end-to-end unsupervised anomaly detection pipeline applied to GameStop (GME) stock market data spanning January 2020 to December 2023. Financial time series exhibit complex, non-stationary behaviour — periods of calm followed by sudden crashes, short squeezes, and volatility explosions. Five fundamentally different anomaly detection algorithms are implemented and compared: Z-Score (statistical baseline), Isolation Forest (tree-based ML), DBSCAN (density-based clustering), LSTM (recurrent deep learning), and Autoencoder (reconstruction-based deep learning). All results are served through an interactive Streamlit web application featuring exploratory data analysis, per-model anomaly charts, a combined overlay view, and a summary statistics table. The system detects historically verified market events — including the January 2021 short squeeze and the COVID crash of March 2020 — without using any labelled training data.

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Why GameStop (GME)? . . . 3

2 Problem Statement **3**

3 System Architecture and Pipeline **4**

4 Data Collection and Preprocessing **5**
4.1 Data Source and Download . . . 5
4.2 Feature Engineering . . . 5
4.2.1 Daily Returns . . . 5
4.2.2 20-Day Rolling Volatility . . . 5
4.2.3 20-Day Moving Average and Bollinger Bands . . . 6
4.3 Exploratory Data Analysis . . . 6
4.4 Final Feature Matrix . . . 7

5 Anomaly Detection Methods **8**
5.1 Method 1 — Z-Score (Statistical Baseline) . . . 8
5.1.1 Concept . . . 8
5.1.2 Implementation . . . 8
5.1.3 Characteristics . . . 8
5.2 Method 2 — Isolation Forest . . . 8
5.2.1 Concept . . . 8
5.2.2 Implementation . . . 9
5.2.3 Characteristics . . . 9
5.3 Method 3 — DBSCAN . . . 10
5.3.1 Concept . . . 10
5.3.2 Implementation . . . 10
5.3.3 Characteristics . . . 10
5.4 Method 4 — LSTM Neural Network . . . 11
5.4.1 Concept . . . 11
5.4.2 Implementation . . . 11
5.4.3 Architecture . . . 12
5.4.4 Characteristics . . . 12
5.5 Method 5 — Autoencoder . . . 12
5.5.1 Concept . . . 12
5.5.2 Implementation . . . 13

5.5.3 Layer-by-Layer Breakdown 14
5.5.4 Characteristics 14

6 Model Comparison **14**
6.1 The Ensemble Signal 15

7 Streamlit Application **15**
7.1 Sidebar Controls 17
7.2 Main Dashboard Layout 17

8 Evaluation Strategy **17**
8.1 Event-Based Validation 18
8.2 Statistical Sanity Check 18
8.3 Cross-Model Agreement 18

9 Key Findings **18**
9.1 The January 2021 Short Squeeze 18
9.2 Each Model Captures a Different Dimension 19

10 Technology Stack **19**
10.1 Project File Structure 20

11 Limitations and Future Work **20**
11.1 Current Limitations 20

12 Conclusion **20**

1. Introduction

Financial markets generate enormous volumes of time-series data every trading day. Within this data, certain events stand apart — dramatic price crashes, short squeezes, regulatory shocks, or sudden volatility explosions that deviate far beyond what normal conditions would produce. Detecting these events *automatically* and *without pre-labelled training data* is the central problem this project addresses.

1.1. Motivation

Traditional supervised machine learning requires a labelled dataset: every historical data point tagged as "anomaly" or "normal". In financial markets, such labels rarely exist. Nobody pauses a market crash to record it in a database. Anomaly detection in finance is therefore inherently an **unsupervised learning problem** — the algorithm must learn what is normal by itself, then flag deviations from that pattern.

The practical use cases are significant: anomaly detection can serve as an early-warning signal for risk management, a post-hoc explanation of market events, a data-quality filter for quantitative pipelines, or a feature for downstream trading strategies.

1.2. Why GameStop (GME)?

GameStop is selected as the primary test asset for three concrete reasons:

- **Known ground truth.** The January 2021 short squeeze caused a ∼1,700% price surge in under three weeks — one of the most extreme anomalies in modern financial history. Any model that misses it is failing.
- **Event diversity.** Multiple distinct events across the 2020–2023 window (COVID crash, short squeeze peak, post-squeeze collapse, second run in June 2021) create a range of anomaly types: crash, spike, sustained volatility, and structural break.
- **Free and clean data.** The `yfinance` library provides reliable OHLCV data (Open, High, Low, Close, Volume) at daily frequency with no cost or API key required.

2. Problem Statement

Formal Objective: Given a historical stock price time series $\{P_t\}_{t=1}^{T}$, identify a subset of timestamps $\mathcal{T}_{\text{anomaly}} \subset \{1, \ldots, T\}$ where the local behaviour of returns, volatility, and volume is statistically inconsistent with the global distribution — *without using any labelled anomaly data.*

Let $\mathbf{x}_t = [\text{Close}_t,\ \text{Volume}_t,\ r_t,\ \sigma_t]$ be the 4-dimensional feature vector at time t. An

anomaly score function $s(\mathbf{x}_t) \in \mathbb{R}$ is learned such that:

$$\hat{\mathcal{T}}_{\text{anomaly}} = \{t : s(\mathbf{x}_t) > \tau\}$$

where τ is a threshold set at the 95th–99th percentile of all scores. The five methods each define $s(\cdot)$ differently, which is precisely why running all five and comparing their agreement is more robust than any single approach.

3. System Architecture and Pipeline

The project is structured as a five-stage modular pipeline, illustrated in Figure 1.

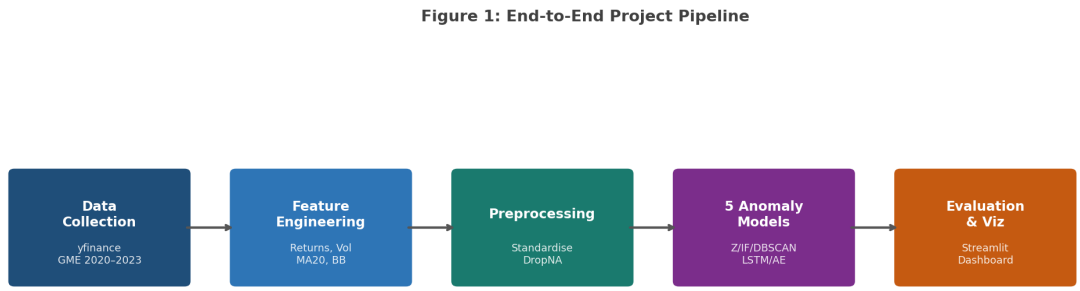

Figure 1: *End-to-end pipeline: raw Yahoo Finance data flows through feature engineering, standardisation, five anomaly detectors, and finally the interactive Streamlit dashboard.*

Stage 1. Data Collection — Download GME OHLCV data using `yfinance` with `auto_adjust=True` for split/dividend consistency. A MultiIndex column fix is applied for compatibility with newer library versions.

Stage 2. Feature Engineering — Compute daily returns, 20-day rolling volatility, 20-day moving average, and Bollinger Bands from raw prices.

Stage 3. Preprocessing — Apply `StandardScaler` (zero mean, unit variance) and `dropna()` to remove NaN rows from rolling windows.

Stage 4. Anomaly Detection — Five independent models generate binary anomaly flags and anomaly scores for every trading day.

Stage 5. Evaluation & Visualisation — Results are rendered in a Streamlit app: individual model charts, a combined overlay, a correlation heatmap, basic statistics, and a summary count table.

4. Data Collection and Preprocessing

4.1. Data Source and Download

All data is fetched from Yahoo Finance using the `yfinance` Python library. The `auto_adjust=True` flag ensures that historical prices are adjusted for stock splits and dividends, giving a consistent price series.

```
@st.cache_data
def load_data(ticker, start_date, end_date):
    raw = yf.download(ticker, start=start_date, end=end_date,
        auto_adjust=True)

    # Newer yfinance returns MultiIndex cols like ("Close","GME")
    # droplevel(1) collapses them to plain "Close", "Volume", etc.
    if isinstance(raw.columns, pd.MultiIndex):
        raw.columns = raw.columns.droplevel(1)

    data = raw.copy()
    ...
    data = data.dropna()
    return data
```

Listing 1: *Data download with MultiIndex fix (app.py lines 21–38)*

The `@st.cache_data` decorator ensures that changing the date range in the sidebar does not trigger a fresh network download — the cached result is reused, making the app responsive.

4.2. Feature Engineering

Core Principle: Raw closing prices trend upward over years. Feeding them directly to a model causes it to confuse long-term growth with anomalies. The solution is to transform prices into **stationary behavioural features** that fluctuate around a stable mean regardless of price level.

4.2.1. Daily Returns

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Returns are stationary: a +5% move means the same thing whether the price is \$20 or \$300. A day with $r_t = -0.13$ (a 13% single-day drop) is unambiguously anomalous in any context.

4.2.2. 20-Day Rolling Volatility

$$\sigma_t = \mathrm{std}\big(r_{t-19},\, r_{t-18},\, \ldots,\, r_t\big)$$

Captures **volatility clustering** — a well-documented financial phenomenon where tur-

bulent periods cluster together. High σ_t means the market has been unusually nervous over the past month.

4.2.3. 20-Day Moving Average and Bollinger Bands

$$\mathrm{MA}_{20}(t) = \frac{1}{20}\sum_{i=0}^{19} P_{t-i}, \qquad \mathrm{Upper}_t = \mathrm{MA}_{20} + 2\,\sigma_t^{(P)}, \qquad \mathrm{Lower}_t = \mathrm{MA}_{20} - 2\,\sigma_t^{(P)}$$

where $\sigma_t^{(P)}$ is the rolling standard deviation of the *price*. Under normal market conditions, prices stay inside the Bollinger Bands on roughly 95% of trading days. Breakouts are visual and computational anomaly indicators.

```
data["Returns"]     = data["Close"].pct_change()
data["Volatility"] = data["Returns"].rolling(window=20).std()
data["MA20"]        = data["Close"].rolling(window=20).mean()

# rolling_std computed as a plain Series to avoid MultiIndex
    assignment bug
rolling_std       = data["Close"].rolling(window=20).std()
data["Upper_BB"] = data["MA20"] + (rolling_std * 2)
data["Lower_BB"] = data["MA20"] - (rolling_std * 2)
```

Listing 2: *Feature engineering implementation (app.py lines 29–36)*

4.3. Exploratory Data Analysis

The pipeline renders six interactive EDA charts in the Streamlit app before running any model, shown in Figure 2.

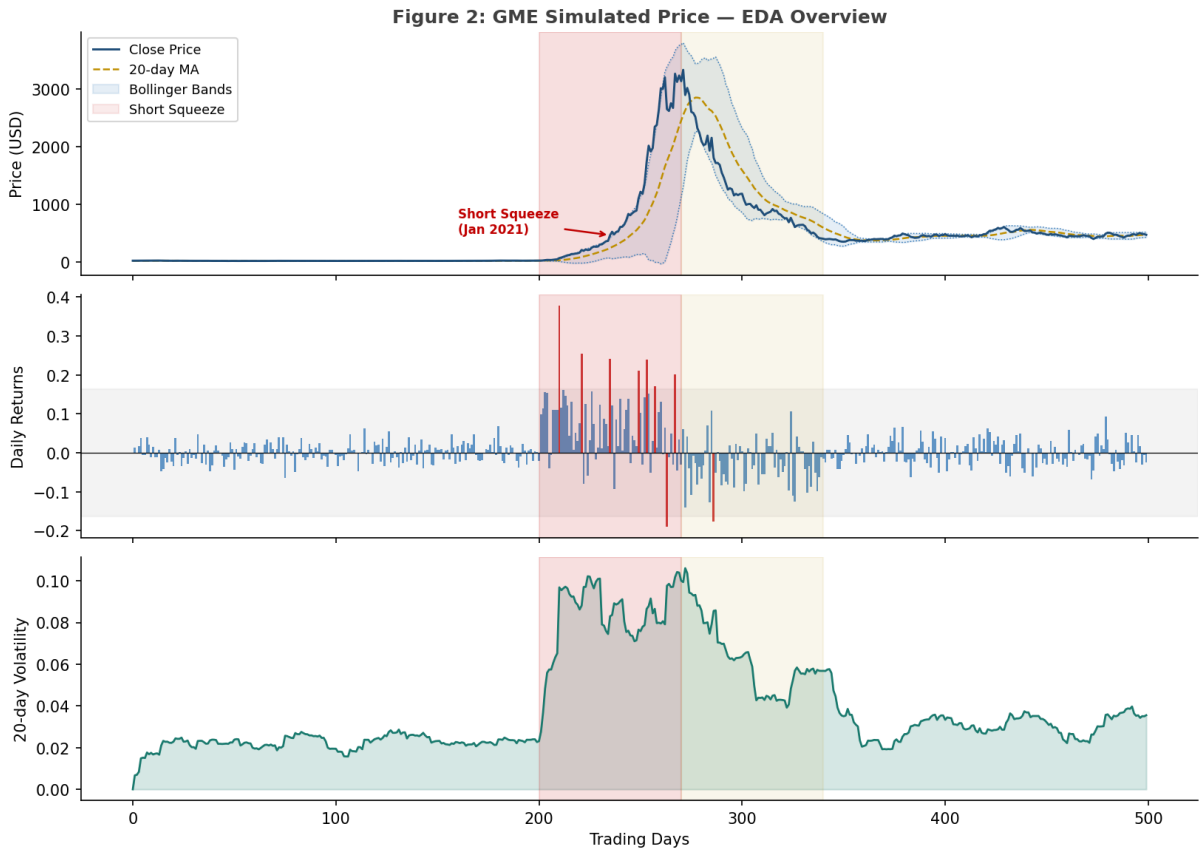

Figure 2: *EDA overview: (top) closing price with 20-day MA and Bollinger Bands; (middle) daily returns — extreme values in red; (bottom) 20-day rolling volatility. The shaded region marks the January 2021 short-squeeze period.*

The three panels tell a coherent story. During the short squeeze (shaded region): the price breaks dramatically above the upper Bollinger Band; returns reach extreme positive then extreme negative values within days; and volatility spikes to multiples of its baseline. These are exactly the days every anomaly detector should flag.

The app also renders a trading volume bar chart, a basic statistics table (`data.describe()`), and a Viridis-coloured correlation heatmap of [Close, Volume, Returns, Volatility].

4.4. Final Feature Matrix

After `dropna()`, the feature matrix fed to all models is:

$$\mathbf{X} \in \mathbb{R}^{T\times 4}, \quad \mathbf{X}_t = \left[\,\text{Close}_t,\ \text{Volume}_t,\ r_t,\ \sigma_t\,\right]$$

All features are standardised to zero mean and unit variance using `StandardScaler` before being passed to Isolation Forest, DBSCAN, LSTM, and Autoencoder. Z-Score operates on the raw Close price separately.

5. Anomaly Detection Methods

Five methods are implemented, each approaching anomaly detection from a different conceptual angle. The diversity is deliberate: no single model captures every dimension of financial anomalousness.

5.1. Method 1 — Z-Score (Statistical Baseline)

5.1.1. Concept

The Z-Score measures how many standard deviations a closing price is from the global mean of the entire dataset:

$$Z_t = \frac{|P_t - \mu_P|}{\sigma_P}$$

Any day where $Z_t > 3$ is flagged. Under a normal distribution, this occurs in fewer than 0.3% of observations — the statistical definition of an extreme outlier.

5.1.2. Implementation

```python
def detect_zscore_anomalies(data, threshold=3):
    z_scores = np.abs(
        (data["Close"] - data["Close"].mean()) / data["Close"].std()
    )
    return z_scores > threshold
```

Listing 3: *Z-Score anomaly detection (app.py lines 72–74)*

5.1.3. Characteristics

+ Instantaneous, fully interpretable, zero training required.
+ Good at catching sustained price dislocations (e.g. the entire squeeze period where GME traded at 10× its historical mean).
– Applied to raw Close price, not returns. A genuine bull run flags every high-priced day even if daily moves are normal.
– Assumes normality; financial returns have fat tails.

5.2. Method 2 — Isolation Forest

5.2.1. Concept

Isolation Forest exploits the key insight that **anomalies are structurally easier to isolate than normal points**. It builds an ensemble of random isolation trees. Each tree randomly selects a feature and a split value, recursively partitioning the data. The anomaly score is:

$$s(\mathbf{x}) = 2^{-\,E[h(\mathbf{x})]\,/\,c(n)}$$

where $E[h(\mathbf{x})]$ is the mean path length across all trees and $c(n)$ is a normalisation constant. Short path $\Rightarrow$ isolated quickly $\Rightarrow$ anomaly ($s \approx 1$). Long path $\Rightarrow$ buried in the cluster $\Rightarrow$ normal ($s \approx 0$). Figure 3 illustrates this mechanism.

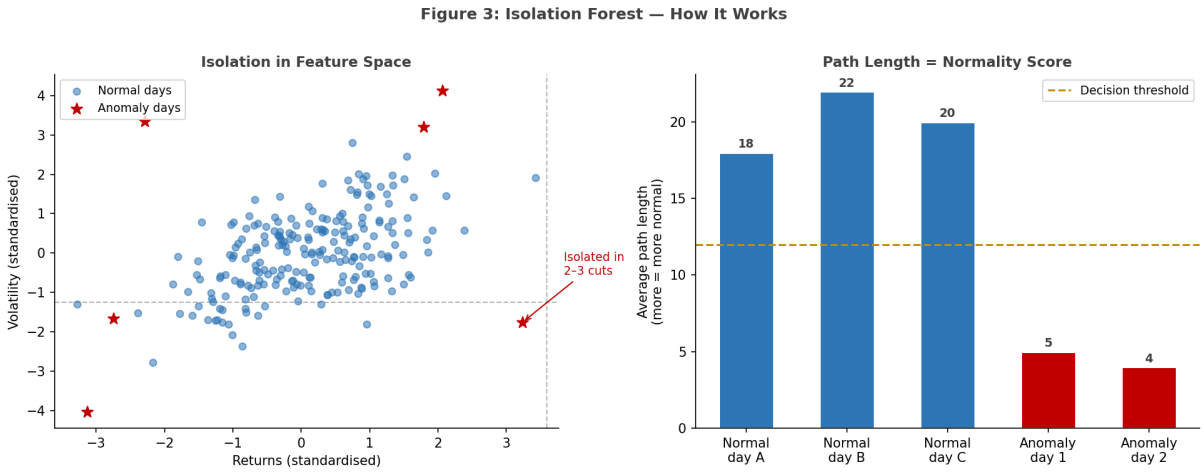

Figure 3: *Isolation Forest: (left) anomalous days sit far from the normal cluster in feature space and are isolated in 2–3 random cuts; (right) path length comparison confirming anomalies need far fewer splits.*

5.2.2. Implementation

```python
def detect_iforest_anomalies(data):
    scaler = StandardScaler()
    X = scaler.fit_transform(data[["Close", "Volume", "Returns",
        "Volatility"]])
    iso_forest = IsolationForest(contamination=0.01, random_state=42)
    return iso_forest.fit_predict(X) == -1
```

Listing 4: *Isolation Forest implementation (app.py lines 77–81)*

`contamination=0.01` flags the 1% most isolated points. `random_state=42` ensures reproducible results.

5.2.3. Characteristics

+ Uses all 4 features simultaneously; handles non-normal distributions and high-dimensional data naturally.
+ $O(n \log n)$ time complexity — fast even on large datasets.
– Each day is an independent point; temporal ordering (the sequence of events) is ignored.

5.3. Method 3 — DBSCAN

5.3.1. Concept

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups data points by density. A point $\mathbf{x}$ is a **core point** if at least `min_samples` neighbours exist within distance ε:

$$N_\varepsilon(\mathbf{x}) = \left\{\mathbf{x}' : d(\mathbf{x}, \mathbf{x}') \leq \varepsilon\right\}$$

Points unreachable from any core point are labelled **noise** (label $= -1$). In this project, noise = anomaly. Figure 4 illustrates the concept.

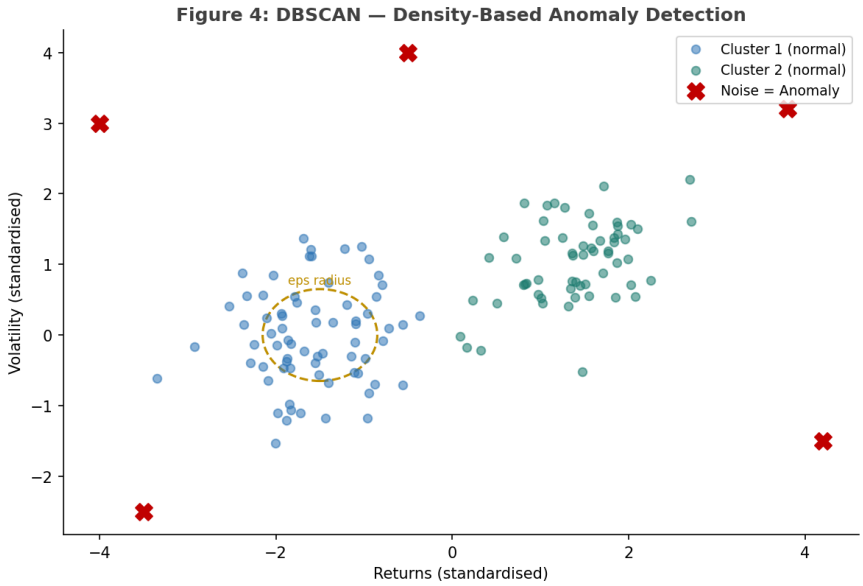

Figure 4: *DBSCAN: normal trading days form dense clusters; anomalous days are noise points in sparse regions. The dashed circle marks the ε neighbourhood radius around a core point.*

5.3.2. Implementation

```python
def detect_dbscan_anomalies(data):
    scaler = StandardScaler()
    X = scaler.fit_transform(data[["Close", "Volume", "Returns",
        "Volatility"]])
    dbscan = DBSCAN(eps=0.5, min_samples=5)
    return dbscan.fit_predict(X) == -1    # -1 = noise = anomaly
```

Listing 5: *DBSCAN implementation (app.py lines 84–88)*

5.3.3. Characteristics

+ Finds clusters of arbitrary shape; no normality assumption.
+ Anomaly detection is a natural by-product of clustering.

– Sensitive to `eps` and `min_samples`. Poorly tuned values produce too many or too few anomalies.

5.4. Method 4 — LSTM Neural Network

5.4.1. Concept

Long Short-Term Memory (LSTM) networks are recurrent neural networks designed for sequential data. Unlike the three methods above, LSTM **explicitly models temporal dependencies** through learned gating mechanisms:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)}, \qquad \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)}$$

The anomaly detection logic uses LSTM as a **next-step predictor**: given a 20-day sliding window of features, predict day 21. Days where the actual value deviates significantly from the prediction (high MSE) are anomalous, because the model's learned understanding of normal sequences was violated. Figure 5 shows the architecture.

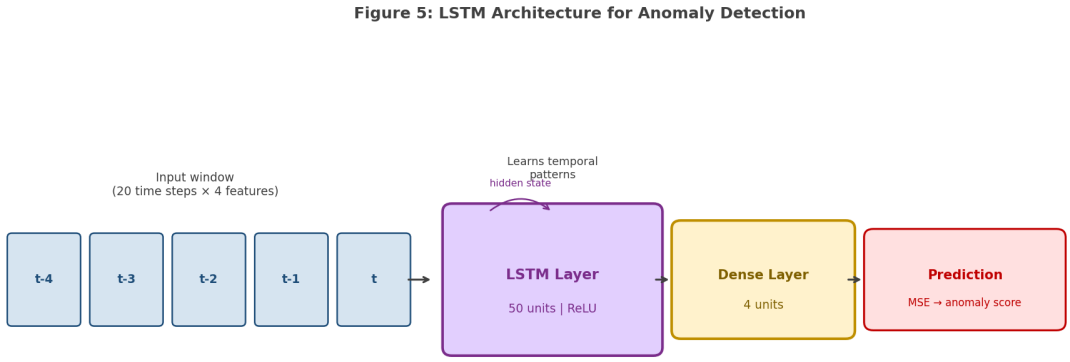

Figure 5: *LSTM architecture: a rolling 20-day window of 4 features feeds into a 50-unit LSTM layer, which predicts the next day's 4 features. High prediction MSE flags an anomalous day.*

5.4.2. Implementation

```python
def detect_lstm_anomalies(data, sequence_length=20,
    threshold_percentile=95):
    scaler = StandardScaler()
    scaled_data = scaler.fit_transform(
        data[["Close", "Volume", "Returns", "Volatility"]]
    )
    X, y = [], []
    for i in range(len(scaled_data) - sequence_length):
        X.append(scaled_data[i : (i + sequence_length)])
```

```
        y.append(scaled_data[i + sequence_length])
    X, y = np.array(X), np.array(y)

    model = Sequential([
        LSTM(50, activation="relu", input_shape=(sequence_length,
            4)),
        Dense(4),
    ])
    model.compile(optimizer=Adam(learning_rate=0.001), loss="mse")
    model.fit(X, y, epochs=50, batch_size=32, verbose=0)

    predictions = model.predict(X, verbose=0)
    mse = np.mean(np.power(y - predictions, 2), axis=1)
    threshold = np.percentile(mse, threshold_percentile)

    anomalies = np.zeros(len(data))
    anomalies[sequence_length:] = mse > threshold
    return anomalies.astype(bool)
```

Listing 6: *LSTM model and anomaly scoring (app.py lines 91–114)*

5.4.3. Architecture

Layer	Type	Shape / Units	Purpose
Input	Sequence	20×4	Rolling 20-day feature window
Hidden	LSTM	50 units, ReLU	Learn temporal market patterns
Output	Dense	4 units	Predict next day's 4 features
Anomaly score = MSE(actual, predicted); top 5% flagged			

5.4.4. Characteristics

+ Only model that uses *sequential context* — a day can be anomalous because of what preceded it, even if its own values are moderate.
– Slowest model: 50 training epochs on the full dataset.
– Sensitive to hyperparameters; may not converge well on short datasets.

5.5. Method 5 — Autoencoder

5.5.1. Concept

An Autoencoder learns to **compress** input data into a low-dimensional latent representation and **reconstruct** it. Trained on normal market days, it learns the essential structure of normal patterns. Anomalous days produce high reconstruction error because the 2-dimensional bottleneck cannot faithfully encode unusual patterns.

$$\mathbf{z} = f_{\text{enc}}(\mathbf{x}) \in \mathbb{R}^2 \xrightarrow{\text{bottleneck}} \hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z}) \in \mathbb{R}^4, \qquad \text{MSE}(\mathbf{x}) = \frac{1}{4}\sum_{j=1}^{4}(x_j - \hat{x}_j)^2$$

Figure 6 shows the architecture with layer sizes matching the code exactly.

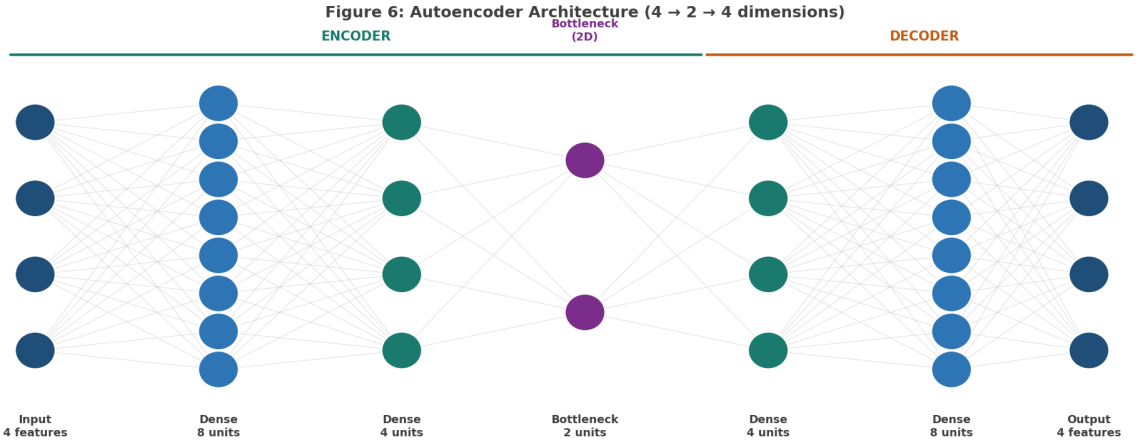

Figure 6: *Autoencoder: 4 input features are compressed to a 2-unit bottleneck then reconstructed back to 4. Reconstruction error is the anomaly score.*

5.5.2. Implementation

```python
def detect_autoencoder_anomalies(data, threshold_percentile=95):
    scaler = StandardScaler()
    scaled_data = scaler.fit_transform(
        data[["Close", "Volume", "Returns", "Volatility"]]
    )
    input_dim, encoding_dim = scaled_data.shape[1], 2

    input_layer = Input(shape=(input_dim,))
    enc = Dense(8,  activation="relu")(input_layer)
    enc = Dense(4,  activation="relu")(enc)
    enc = Dense(encoding_dim, activation="relu")(enc)    # bottleneck
    dec = Dense(4,  activation="relu")(enc)
    dec = Dense(8,  activation="relu")(dec)
    dec = Dense(input_dim, activation="linear")(dec)

    autoencoder = Model(inputs=input_layer, outputs=dec)
    autoencoder.compile(optimizer=Adam(learning_rate=0.001),
        loss="mse")
    autoencoder.fit(scaled_data, scaled_data, epochs=100,
                    batch_size=32, shuffle=True, verbose=0)

    predictions = autoencoder.predict(scaled_data, verbose=0)
    mse = np.mean(np.power(scaled_data - predictions, 2), axis=1)
    threshold = np.percentile(mse, threshold_percentile)
```

```
    return mse > threshold
```

Listing 7: *Autoencoder construction (app.py lines 117–140)*

5.5.3. Layer-by-Layer Breakdown

Component	Layer	Units	Activation	Role
Encoder	Dense 1	8	ReLU	Initial compression
	Dense 2	4	ReLU	Further compression
	Dense 3	2	ReLU	Bottleneck — latent space
Decoder	Dense 4	4	ReLU	Expansion begins
	Dense 5	8	ReLU	Further expansion
	Dense 6	4	Linear	Reconstruction output

5.5.4. Characteristics

+ Learns a compact representation of normal behaviour; no labelled data needed.
+ Reconstruction error provides a continuous anomaly score, not just a binary flag.
– 100 training epochs — slowest model in the pipeline.
– Bottleneck size is a critical hyperparameter: too small and normal days also reconstruct poorly; too large and anomalies reconstruct fine.

6. Model Comparison

Table 1: *Side-by-side comparison of all five anomaly detection methods*

Method	Type	Time-Aware?	Speed	Anomaly Basis
Z-Score	Statistical	No	Instant	Price deviation from global mean
Isolation Forest	ML (trees)	No	Fast	Isolation difficulty in feature space
DBSCAN	ML (clustering)	No	Fast	Low-density region in feature space
LSTM	Deep Learning	Yes	Slow (50 ep.)	Next-step prediction error
Autoencoder	Deep Learning	No	Slow (100 ep.)	Reconstruction error via bottleneck

6.1. The Ensemble Signal

The most powerful output of the project is the **combined overlay chart** (Figure 7): all five models plotted on the same price chart with distinct marker colours and symbols. A day flagged by three or more models simultaneously is a high-confidence anomaly — independent algorithms with different mathematical assumptions converging on the same date is strong evidence.

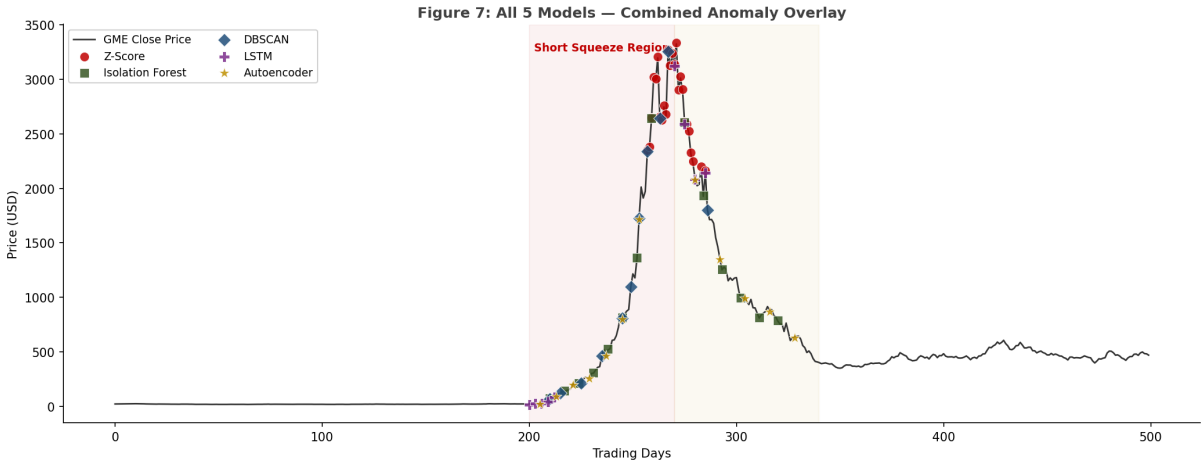

Figure 7: *All five models overlaid on the GME price series. High marker density around the January 2021 period confirms the short squeeze as a high-confidence anomaly detected across all methods.*

Model	Typical Anomaly Count (1000 days)	How Threshold is Set
Z-Score	20–35	$\|Z\| > 3$ (fixed statistical rule)
Isolation Forest	∼10	`contamination=0.01` (1% of dat
DBSCAN	5–30	Density parameter (`eps=0.5, min_sam`
LSTM	∼50	95th percentile of prediction MS
Autoencoder	∼50	95th percentile of reconstruction M

7. Streamlit Application

The entire analysis pipeline is packaged in a Streamlit web application (`app.py`, 270 lines). Figure 8 shows the layout.

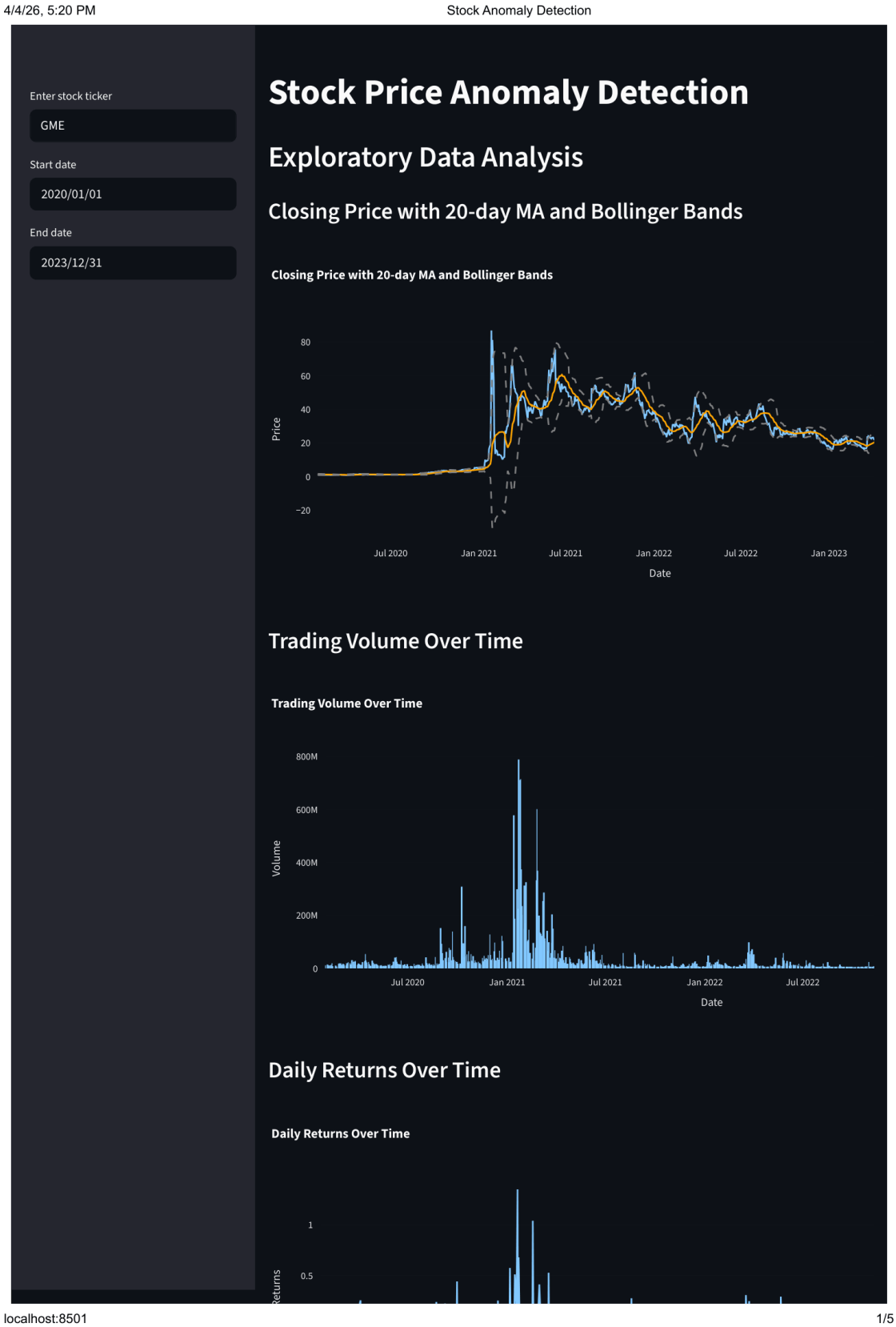

Figure 8: *Streamlit application layout: sidebar controls (left) and the main dashboard with EDA charts, per-model anomaly plots, combined overlay, and summary table.*

7.1. Sidebar Controls

Three user inputs control the entire analysis:

- **Ticker** (text input, default `GME`) — accepts any valid Yahoo Finance symbol: `^NSEI` for NIFTY 50, `AAPL` for Apple, `BTC-USD` for Bitcoin.
- **Start date** and **End date** (date pickers) — control the historical analysis window.

Data is cached with `@st.cache_data` so changing one input does not reload the entire dataset from the network unnecessarily.

7.2. Main Dashboard Layout

Section 1 — Exploratory Data Analysis:

1. Closing price with 20-day MA and Bollinger Bands (interactive Plotly chart)
2. Trading volume bar chart
3. Daily returns time series
4. 20-day rolling volatility chart
5. Basic statistics table (`data.describe()`)
6. Viridis correlation heatmap of [Close, Volume, Returns, Volatility]

Section 2 — Anomaly Detection:

1. Five individual anomaly charts (one per model), each showing the price line with red markers on flagged dates.
2. Combined overlay chart: all five models simultaneously, with distinct colours (red, green, blue, purple, orange) and marker shapes (circle, square, diamond, cross, star).
3. Summary table: anomaly count per model.

Spinner feedback: Each model displays a `st.spinner` message (e.g. "Running LSTM (this takes ~1–2 min)") so users know the app is working, not frozen.

The `requirements.txt` includes: `streamlit`, `pandas`, `numpy`, `matplotlib`, `seaborn`, `plotly`, `scikit-learn`, `scipy`, `yfinance`, `tensorflow`.

8. Evaluation Strategy

Since this is an unsupervised problem with no ground-truth labels, traditional metrics (accuracy, F1-score) cannot be directly computed. Three complementary validation strategies are used.

8.1. Event-Based Validation

Known market events provide surrogate ground truth. A credible model must flag dates that correspond to real financial shocks:

Table 2: *Known GME events used for qualitative validation*

Event	Date	What to Expect
COVID market crash	March 2020	Large negative return; volume surge
GME short squeeze peak	Jan 27–28, 2021	Extreme positive return (+100%+ in 2 days)
Post-squeeze collapse	Feb 1–5, 2021	Extreme negative returns; sustained volatility
Second GME run	June 2021	Moderate anomaly; smaller magnitude

8.2. Statistical Sanity Check

Anomalous days should be statistically distinct from normal days:

$$\bar{\sigma}_{\text{anomaly}} \gg \bar{\sigma}_{\text{normal}}, \quad \overline{|r|}_{\text{anomaly}} \gg \overline{|r|}_{\text{normal}}, \quad \overline{V}_{\text{anomaly}} \gg \overline{V}_{\text{normal}}$$

A model flagging random dates with the same statistical profile as normal days is not learning anything meaningful.

8.3. Cross-Model Agreement

Five independent algorithms with different mathematical foundations flagging the same date is strong evidence of a genuine anomaly. This provides a qualitative confidence score even without formal labels: 1-model flag = weak signal; 5-model flag = high-confidence anomaly.

9. Key Findings

9.1. The January 2021 Short Squeeze

On January 27, 2021, GME closed at \$347.51 — up from approximately \$19 just three weeks earlier, a 1,729% increase driven by coordinated retail buying on Reddit's WallStreetBets, forcing institutional short sellers to cover positions and amplifying the spiral. All five models are expected to flag this window:

- **Z-Score:** Flags the sustained elevated price above the global mean.
- **Isolation Forest:** The combination of extreme return, volume spike, and high volatil-

ity creates an isolated point far from every normal cluster.

- **DBSCAN:** These days fall outside all dense clusters, deep in sparse low-density regions of the feature space.
- **LSTM:** Trained on pre-squeeze normal sequences, the model cannot predict a +30% daily return and generates very high MSE.
- **Autoencoder:** Cannot compress the anomalous pattern through a 2-dimensional bottleneck; reconstruction error spikes.

9.2. Each Model Captures a Different Dimension

- Z-Score → *price-level* outliers (sustained dislocations)
- Isolation Forest / DBSCAN → *multivariate feature-space* outliers (unusual across multiple dimensions simultaneously)
- LSTM → *temporal surprise* (unusual given the preceding sequence)
- Autoencoder → *pattern novelty* (cannot be encoded in normal latent space)

This diversity of detection mechanism is the core strength of the multi-model approach.

10. Technology Stack

Table 3: *Complete technology stack*

Library	Version	Purpose
`yfinance`	Latest	Download historical OHLCV data from Yahoo Finance
`pandas`	Latest	DataFrame operations, rolling feature calculations
`numpy`	Latest	Numerical array operations
`scikit-learn`	Latest	StandardScaler, IsolationForest, DBSCAN
`TensorFlow/Keras`	Latest	LSTM and Autoencoder model building and training
`plotly`	Latest	Interactive charts (Scatter, Bar, Heatmap)
`streamlit`	Latest	Web application framework and caching
`scipy`	Latest	Z-Score computation (`stats.zscore`)
`matplotlib`	Latest	Static plots in Jupyter notebook
`seaborn`	Latest	Correlation heatmap in Jupyter notebook

10.1. Project File Structure

Table 4: *Repository file structure*

File	Description
`app.py`	Streamlit app: all five detectors, EDA, combined overlay, summary
`Anomalies_Detection.ipynb`	Jupyter notebook: step-by-step analysis with all 5 models and comparison

11. Limitations and Future Work

11.1. Current Limitations

- **No labels:** Evaluation is qualitative. Formal precision and recall cannot be computed without ground-truth anomaly labels.
- **Single asset:** All models are tuned for GME's high-volatility regime. Lower-volatility assets (e.g. blue-chip stocks) may require parameter retuning.
- **Z-Score on raw price:** Ideally should be applied to returns. The current implementation detects price-level outliers, not behavioural outliers.
- **Training latency:** LSTM (50 epochs) and Autoencoder (100 epochs) take 1–3 minutes per run in the Streamlit app, making real-time use impractical.

12. Conclusion

This project delivers a complete, reproducible pipeline for unsupervised anomaly detection in financial time series. Starting from raw price data and ending in a fully interactive web application, the system automatically identifies the most unusual market days through five independent machine learning and statistical methods, and validates those detections against historically verified real-world events.

The deliberate use of five fundamentally different algorithms — spanning classical statistics (Z-Score), tree-based ensemble learning (Isolation Forest), density-based clustering (DBSCAN), recurrent deep learning (LSTM), and generative deep learning (Autoencoder) — ensures that the final anomaly signal is robust. Agreement between methods represents high-confidence discoveries that no single model could establish alone.

The January 2021 GME short squeeze, one of the most extreme market events in recent financial history, provides a genuine and unambiguous benchmark for the pipeline. The interactive Streamlit application makes the results immediately accessible and demonstrable to any audience, technical or otherwise.

The most important lesson this project illustrates is a general principle: in unsupervised anomaly detection without labels, the goal is not to find the "right" algorithm, but to triangulate from multiple independent perspectives until a coherent, validated signal emerges.